WathiqNet: A Web-Based Network Security Monitoring System

A PROJECT REPORT SUBMITTED

IN PARTIAL – FULFILMENT OF THE REQUIREMENT

FOR THE AWARD OF THE DEGREE

FOR

BACHELOR IN INFORMATION TECHNOLOGY

Students

Name	ID
Rasha Mohammed Jaber	202307939
Malak Yahya Hassan	202307573
Ebtihal Mohammed Qarawashi	202305512
Wahaj Ahmed Al-Sabi	202304058
Jawahir Mousa Tawhri	202308033
Alhanouf Abdullah Yahya	202306265

SUPERVISOR

Dr. Shazia Ali Syed Ali

COLLEGE OF ENGINEERING & COMPUTER SCIENCE

DEPARTMENT OF COMPUTER SCIENCE

JAZAN UNIVERSITY, JAZAN (KSA)

(Semester – 1)

2025 / 2026

DEPARTMENT OF COMPUTER SCIENCE

APPROVAL FOR BINDING OF GRADUATION PROJECT REPORT

GRADUATION PROJECT (ITEC 425)

GROUP 2514

SESSION 2025 / 2026

To be filled in by student

Student name(s)	:	Rasha Mohammed Jaber - Malak Yahya Hassan - Ebtihal Mohammed Qarawashi Wahaj Ahmed Al-Sabi - Jawahir Mousa Tawhri - Alhanouf Abdullah Yahya
University ID(s).	:	202307939 - 202307573 - 202305512 202304058 - 202308033 - 202306265
Project title	:	**WathiqNet: A Web-Based Network Security Monitoring System**

To be filled in by supervisor

This is to certify that the project submitted and presented by the above mentioned student(s) is

☐ **COMPLETE & ACCEPTED** ☐ **INCOMPLETE** ☐ **NOT ACCEPTED**

and the draft of graduation project report has been corrected from all content flaws, typing errors and language mistakes.

Supervisor name	:	**Dr. Shazia Ali Syed Ali**
Signature	:	
Date	:	

For Departmental Office use only

Date received	:	
Signatures	:	

Preface

This project represents the culmination of the knowledge, skills, and practical experience acquired throughout our academic journey in Software Engineering. It embodies our commitment to applying theoretical concepts to real-world challenges and delivering a practical solution that addresses the needs of efficient network management systems.

The development of this system involved extensive research, analysis, design, and implementation efforts. Throughout the project, we aimed to follow industry standards and best practices, ensuring that the final product is both functional and scalable. The pro- cess enhanced our understanding of system development methodologies and strengthened our ability to collaborate effectively, analyze user requirements, and transform them into a fully operational software solution.

We hope that this work not only fulfills academic requirements but also serves as a meaningful contribution to the field, demonstrating the potential of technology to optimize operations and improve efficiency. Completing this project marks an important milestone in our professional development, and we look forward to applying the lessons learned in future endeavors.

Acknowledgments

First and foremost, we express our sincere gratitude to Allah, the Almighty, for granting us the strength and determination to complete this project successfully.

We extend our heartfelt thanks to Jazan University, particularly the College of Engineering & Computer Science, Department of Computer Science, for providing the academic resources, knowledge, and facilities necessary to carry out this work.

We are deeply grateful to our supervisor, Dr. Shazia Ali Syed Ali, for her continuous guidance, constructive feedback, and encouragement throughout the project. Her support has been pivotal in shaping both the direction and quality of our work.

We also acknowledge our colleagues and peers who contributed valuable ideas, suggestions, and assistance during the project. Their cooperation and support have enriched our learning experience.

Finally, we thank our families and friends for their unwavering encouragement and support. Their motivation has been essential to achieving this milestone, making this project a collective accomplishment supported by many.

ABSTRACT

The increasing reliance on the internet and local networks for education, business, and personal use has made cybersecurity a critical concern. While large organizations invest heavily in security, small businesses, institutions, and individuals often remain vulner- able due to limited resources and technical knowledge. To address this challenge, this project introduces WathiqNet, a web-based network security monitoring system designed to provide simple yet effective tools for detecting unusual activities within a network. WathiqNet focuses on detection and reporting. It collects network activity data, applies detection rules to identify suspicious behavior, and presents the results on a user-friendly website. The system issues real-time alerts and compiles weekly reports that highlight security incidents, classify their severity, and provide suggestions for improvement. This approach makes network security more accessible to students, small offices, and home users. The project aims to demonstrate that even with limited resources, it is possible to build a practical and usable security monitoring tool that raises awareness and contributes to safer digital environments.

KEYWORDS: Network Security, Monitoring, Web Application, Alerts, Reports, Detection.

Contents

List of Tables

List of Figures

Chapter 1

INTRODUCTION

Technology and the internet have become essential for almost all aspects of modern life. Businesses rely on networks to manage operations, students use them for research and collaboration, and households depend on them for communication and entertainment. However, this increasing dependency has brought new risks, as malicious actors target networks with a variety of attacks. For small environments such as schools, clinics, and homes, these risks are real but often ignored due to cost and complexity of professional security systems.[1]

This project proposes WathiqNet, a website-based monitoring system that addresses this problem. WathiqNet is not a replacement for enterprise security solutions, but it offers a practical way for smaller users to gain visibility into their networks, detect possible threats, and learn more about cybersecurity in an accessible manner. By providing clear alerts, simple reports, and an intuitive interface, the system empowers non-expert users to become more aware of network security issues.

The goal of WathiqNet is not only technical but also educational: it provides a platform for students and small organizations to understand how monitoring works and why it is important. In doing so, it contributes to building a culture of awareness and responsibility in digital environments.

1.1 Project Overview

WathiqNet is a prototype system that demonstrates how a website can be used to monitor and display network activities. It works by collecting logs of network usage and analyzing them using predefined rules. When the system detects unusual or suspicious patterns, it issues alerts and records them in a database. A user-friendly dashboard then allows users to:

- View network statistics and connected devices.
- Receive alerts about abnormal activities.

- Access weekly reports that summarize incidents, threat levels, and recommendations.

The system is built around simplicity and clarity. Users do not need advanced technical skills to use WathiqNet; the dashboard is designed with accessibility in mind. The focus is on providing meaningful information in a clear format rather than overwhelming users with complex technical details.

1.2 Problem Statement

The development of WathiqNet is driven by several key problems observed in small networks and non-technical environments:

- **Lack of simple monitoring tools:** Most available network security solutions are designed for professionals and require advanced configuration, making them unsuitable for beginners or small organizations.
- **Limited technical expertise:** Users in schools, small offices, and home environments often lack the knowledge to understand complex logs or operate enterprise-grade IDS/IPS tools.
- **High complexity of existing systems:** Tools such as Zeek, Snort, and Suri- cata offer powerful capabilities but require command-line skills, rule tuning, and additional dashboards, which increase operational difficulty.
- **Insufficient visibility into network activity:** Many small networks run without any form of monitoring, leading to unnoticed suspicious activities or early signs of cyber threats.
- **No accessible reporting mechanisms:** Existing tools rarely provide simple weekly summaries or beginner-friendly threat explanations, making it hard for non-experts to interpret events.
- **Resource constraints:** Professional monitoring systems require powerful hardware or dedicated servers, which small environments often cannot afford.
- **Educational gap in cybersecurity awareness:** Students and basic users lack practical learning tools that demonstrate how monitoring and detection work in real networks.

1.3 Project Goals and Objectives

1.3.1 Goals

- To design a web-based monitoring system that increases awareness about network security.
- To provide an accessible tool for small organizations and individuals.
- To demonstrate how detection and reporting can be achieved within limited resources.

1.3.2 Objectives

1. Develop a website with a dashboard that displays network activity, alerts, and reports.
2. Implement detection rules to identify suspicious behavior.
3. Provide real-time alerts through the website interface.
4. Generate automated weekly reports that classify incidents by severity and offer recommendations.
5. Ensure that the system remains simple and understandable for non-expert users.
6. Test the system using controlled scenarios and sample data to validate its functions.

1.4 Project Scope

The scope of WathiqNet is carefully defined to ensure that it is achievable within the limits of an academic project.

1.4.1 In Scope:

- Monitoring network activity and logging data.
- Identifying suspicious activities through predefined detection rules.
- Developing a website dashboard that shows alerts and statistics.

- Creating a reporting module that generates weekly summaries.
- Testing the system with small-scale networks and sample traffic.

1.4.2 Out of Scope:

- Advanced enterprise-level features such as machine learning-based detection or integration with commercial security systems.
- Active prevention and blocking of attacks (Intrusion Prevention). WathiqNet is focused on monitoring and reporting.
- Large-scale deployments that handle thousands of devices or high-speed enterprise networks.

1.4.3 Boundaries:

- The system is intended for small businesses, student projects, or home use.
- The main platform is a web application accessible through common web browsers.

1.5 Limitations and Constraints

Like any academic project, WathiqNet comes with certain limitations:

- **Accuracy Limitations:** Since detection is based on rules, it may miss complex attacks or generate false alerts.
- **Performance Limitations:** The system is not designed for high-traffic networks and may not handle very large datasets efficiently.
- **Time Constraints:** The project must be completed within the academic schedule, limiting the extent of testing and optimization.
- **Knowledge Limitations:** Users may need some basic understanding of alerts and security terms to interpret the results correctly.
- **Resources:** The project will be implemented using commonly available tools and frameworks, avoiding costly or advanced technologies.

1.6 Assumptions

To ensure that the project can be completed successfully, the following assumptions are made:

1. The network environment where WathiqNet is tested allows for traffic monitoring and log collection.
2. Users have access to a web browser to use the system's dashboard.
3. Internet connectivity will be available for system updates and testing purposes.
4. Users are willing to review alerts and reports regularly.
5. Sample datasets or controlled test scenarios will be provided for validating the system's features.
6. The project team will have access to the necessary development tools, including a database, web framework, and testing environment.[2]

Chapter 2

PREVIOUS WORKS / LITERATURE REVIEW

The growth of Network Security Monitoring (NSM) has produced a spectrum of tools ranging from packet analyzers and rule-based intrusion detection systems (IDS) to full NSM platforms. Most established solutions prioritize depth, flexibility, and enterprise scalability. However, these strengths often come with complexity, hardware demands, and steep learning curves that can be challenging for small offices, home labs, and student projects. WathiqNet positions itself as a lightweight, educationally oriented, web-based monitoring and reporting system that privileges clarity and ease of use over exhaustive feature sets. It focuses on near real-time alerting, weekly summaries, and actionable recommendations suitable for environments with limited resources and expertise.

2.1 Comparable Systems

2.1.1 Zeek (formerly Bro)

A powerful network analysis framework that extracts rich metadata and enables custom scripts.[3]

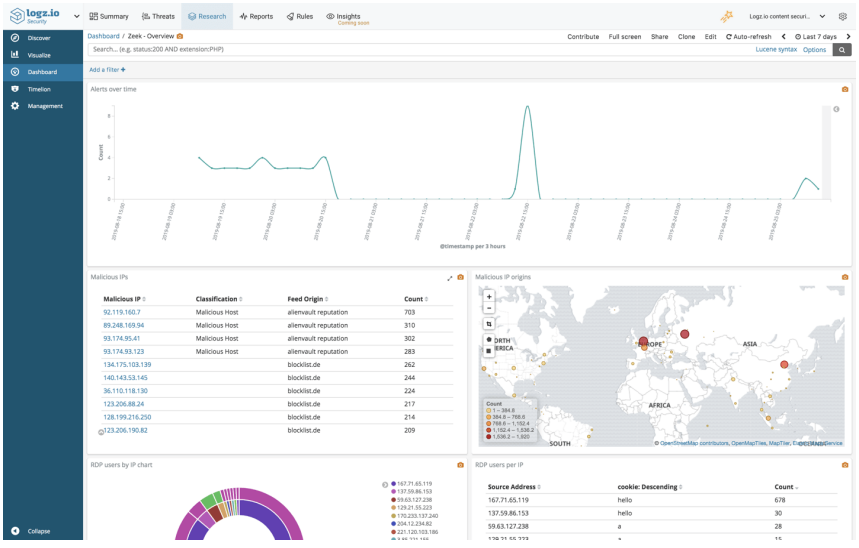


Figure 2.1: Zeek Dashboard

2.1.2 Snort (IDS)

Signature-based IDS widely used for detecting known threats via rule sets (e.g., emerging threats).[4]

Mature ecosystem, extensive community rules, and proven detection of known attack patterns.

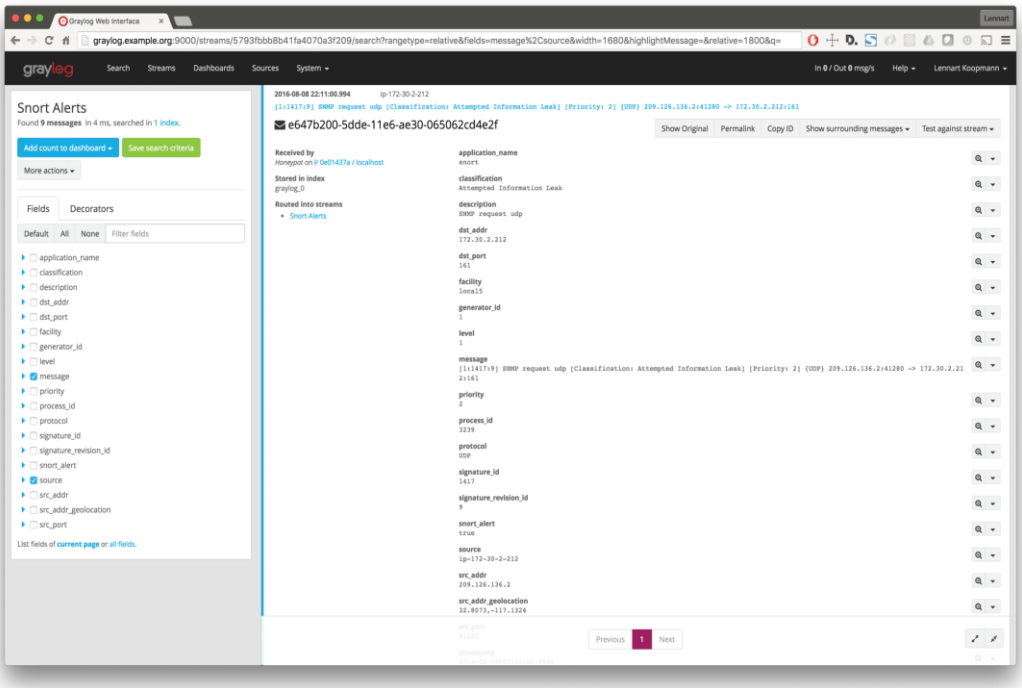


Figure 2.2: Snort — An Intrusion Detection System

Limitations: rule curation and false positive management demand ongoing effort; minimal native reporting/UX; scaling or inline prevention adds complexity and hardware requirements.

2.1.3 Suricata (IDS/IPS)

Multithreaded IDS/IPS with high performance, protocol detection, and file extraction.

Strong for modern throughput and supports many Snort-style rules.

Limitations: optimal performance and accuracy require tuning; dashboards are external (ELK, Kibana) adding setup overhead; noise reduction and weekly managerial reports are not first-class features.

2.1.4 Security Onion (NSM distro)

An integrated NSM platform bundling Zeek, Suricata, sensor management, and rich dashboards.[5]

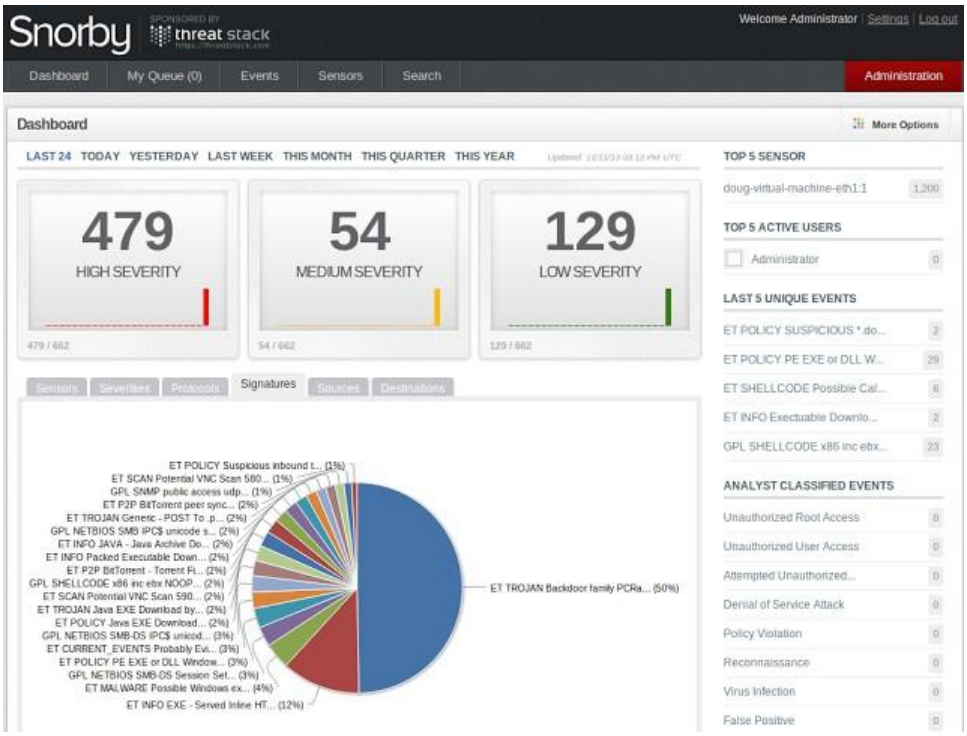


Figure 2.3: Security Onion (NSM distro)

Excellent for enterprise-grade visibility and investigations.

Limitations: heavyweight to deploy/maintain; assumes dedicated hardware and admin

skill; overkill for small networks seeking simple alerts plus weekly summaries.

2.2 WathiqNet Advantages

- **Simplicity first UX:** A web dashboard designed for non-experts—clear incident cards, plain language labels, and minimal configuration to get value quickly.
- **Educational orientation:** Exposes core NSM concepts (alerts, severity, incident types) with explanations and suggestions, supporting learning outcomes for students and small teams.
- **Lightweight footprint:** Designed for modest traffic and commodity hardware; avoids heavyweight stacks (e.g., full ELK) unless explicitly required.
- **Built-in reporting cadence:** Automated weekly reports that aggregate incidents, classify severity, and recommend actions—no extra data pipeline assembly needed.
- **Focused scope (monitoring & reporting):** Reduces cognitive load by avoiding IPS/active blocking; users get visibility without risking accidental traffic disruption.
- **Opinionated defaults:** Predefined detection rules and severity mapping provide immediate signal; users can gradually refine rather than start from a blank slate.
- **Accessible deployment:** Web-based app with common dependencies; straightforward setup for classrooms, labs, and small offices.
- **Alert hygiene:** Emphasis on concise, contextual alerts to reduce noise; highlights "what happened," "why it matters," and "what to consider next."
- **Privacy aware posture:** Encourages capturing only necessary telemetry for educational/SMB use, limiting over-collection.
- **Extensibility path:** Modular design to plug in additional rules or external log sources later without re-architecting the system.

2.3 Comparative Table

Feature	WathiqNet	Zeek	Snort	Suricata	Security Onion
Primary Focus	Simple monitoring & reporting	Deep network analysis	Signature IDS	Multithreaded IDS/IPS	Full NSM platform
Ease of Onboarding	High (web UI)	Low–Medium	Medium	Medium	Low
Hardware Needs	Low	Medium	Medium	Medium–High	High
Built-in Reporting	Weekly summaries	Logs + external	Basic alerts	Basic alerts	Rich dashboards
False Positives	Opinionated defaults	Requires tuning	Rule hygiene	Rule hygiene	Complex tools
Educational	High	Medium	Medium	Medium	Medium
IPS Capability	No	No	Optional	Yes	Possible
Best Fit	Students/SMBs	Researchers	Known threats	High throughput	Enterprise

Table 2.1: Comparison of WathiqNet with Existing Systems

2.4 Conclusion

The NSM ecosystem offers powerful, battle-tested platforms, but many are optimized for enterprises or skilled operators. For small environments seeking clarity over complexity, WathiqNet provides a purposefully scoped alternative: an approachable web interface, real-time alerts, and scheduled, plain-language weekly reports that translate technical signals into actionable understanding. This balance of simplicity, guidance, and visibility addresses a gap between DIY packet tools and heavyweight enterprise stacks, making network security monitoring more accessible and sustainable for beginners and resource-constrained teams.

Chapter 3

FEASIBILITY STUDY

This section evaluates the viability of implementing the WathiqNet system by analyzing economic, technical, and functional aspects. It examines whether the proposed solution can be successfully developed, deployed, and maintained within available resources, time, and expertise.

3.1 Purpose of The Feasibility Study

The primary purpose of this feasibility study is to evaluate if WathiqNet can be realistically developed and sustained as an effective web-based network security monitoring system. It identifies the practical constraints, cost-benefit considerations, and potential technical challenges. The study ensures that the project's scope, objectives, and timeline align with the available resources and the capabilities of the development team. Additionally, it aims to determine the educational and practical value of the system for students, small businesses, and home users seeking accessible network security awareness tools.

The feasibility study acts as a decision-making guide to confirm that WathiqNet's design and functionality can be achieved within academic limitations while producing measurable benefits for non-expert users.

3.2 Justification For The Proposed System

The need for WathiqNet arises from the increasing reliance on networked systems and the growing cybersecurity threats that affect small organizations and individuals. While large enterprises employ advanced monitoring tools, small users lack affordable and understandable solutions. Many existing platforms, such as Zeek or Snort, require specialized knowledge and complex configuration, making them unsuitable for non-technical users.

WathiqNet addresses this gap by offering an easy-to-use, web-based monitoring platform focused on real-time alerts and weekly reports. It simplifies threat visibility, enhances awareness, and provides educational insights into network security. The system's justifica-

tion lies in its accessibility, educational utility, and ability to provide essential monitoring functions without requiring costly infrastructure or advanced expertise. By combining simplicity with functionality, WathiqNet democratizes cybersecurity monitoring for small environments.

3.3 Economic Feasibility

WathiqNet is designed to be cost-effective, leveraging open-source tools and minimal hardware resources.

- **Development Costs:** The system will use open-source frameworks such as Python/Flask, PHP/Laravel, or Node.js for backend development, and standard web technologies (HTML, CSS, JavaScript) for the frontend. This minimizes software licensing costs.
- **Hardware Requirements:** Runs efficiently on standard PCs or small servers; no need for enterprise-level hardware. It can even operate within virtual environments or educational labs.
- **Operational Costs:** Maintenance is minimal, requiring only periodic updates to detection rules and regular report generation. Cloud hosting can be used for small monthly fees if remote access is desired.
- **Benefit Analysis:** The system's low-cost setup delivers significant benefits such as network visibility, security awareness, and reduced risks of undetected intrusions.
- **Return on Investment (ROI):** In academic and small business contexts, the time saved from manual log inspection and the reduction in potential downtime or data loss provide strong ROI.

The project is economically feasible, as it offers high value at a low cost by reusing open technologies and focusing on essential monitoring features rather than advanced enterprise capabilities.

3.3.1 Project Risks

- Detection Inaccuracy: Possible false positives or missed threats due to rule-based detection.
- Data Quality Issues: Logs may be incomplete or inconsistent.

- Security Risks: The web application itself may be targeted if not properly secured.
- User Misinterpretation: Non-technical users may misunderstand alerts.
- Time Limitations: Academic deadlines may restrict deep testing.

3.3.2 Project Costs

- Development Cost: Very low, mainly team effort; uses open-source tools.
- Hardware Cost: Requires only a basic PC or small server.
- Operational Cost: Optional low-cost hosting if deployed online.
- Maintenance Cost: Periodic updates to rules and system patches.

3.3.3 Project Challenges

- Creating Accurate Rules: Balancing sensitivity and accuracy.
- Simple Data Visualization: Converting technical data into easy dashboards.
- Handling Log Volume: Ensuring smooth performance with growing logs.
- Security Hardening: Protecting the system from common web vulnerabilities.
- Scalability Limits: Designed for small networks, not large environments.

3.4 Technical Feasibility

Technical feasibility examines whether the proposed system can be developed using the available technologies and expertise.

- **Technology Stack:** The system can be implemented using web-based frameworks (Laravel, Django, Node.js) connected to a relational database (MySQL). Detection logic can rely on open-source libraries for log parsing and rule-based analysis.
- **Integration Capability:** WathiqNet can collect logs from network devices, routers, or test environments using simple APIs or file imports. It is compatible with structured log formats such as CSV and JSON.

- **Performance Expectations:** The system is designed for small-scale use, supporting up to a few dozen active devices. Data summarization and weekly batching reduce computational overhead.
- **Security Considerations:** Basic authentication and role-based access control (admin/viewer) are integrated. HTTPS and secure database connections ensure safe data handling.
- **Maintenance & Scalability:** Modular design allows the system to evolve with additional detection rules or visualization components. The simplicity of the architecture ensures that new users can deploy and maintain it easily.

Potential technical risks, such as false positives or inaccurate rule detection, can be mitigated through continuous testing and feedback refinement. Therefore, the technical feasibility of WathiqNet is strong, supported by mature, well-documented, and accessible development technologies.

3.5 Desired System Functionality

WathiqNet aims to offer clear, actionable insights into network security events through a simplified web interface. The main functionalities include:

1. **Network Monitoring:** Collect and store network activity logs, providing a centralized view of devices and traffic patterns.
2. **Threat Detection:** Apply rule-based detection mechanisms to identify suspicious or abnormal behaviors, classify them by severity, and trigger alerts.
3. **Alert Management:** Display alerts on a dashboard with time, source, description, and recommended action. Allow users to acknowledge and resolve alerts.
4. **Weekly Reporting:** Automatically generate comprehensive weekly reports summarizing incidents, their frequency, and actionable security recommendations.
5. **Dashboard Visualization:** Provide user-friendly charts, tables, and summaries to make complex data easily understandable for non-technical users.
6. **User Management:** Support multiple roles—administrators for configuration and viewers for monitoring—to encourage shared use in classrooms or small offices.

7. **System Configuration:** Allow users to adjust detection thresholds, enable/disable rules, and customize report templates.
8. **Data Backup & Export:** Enable exporting reports and logs for archival or academic submission.

WathiqNet is economically and technically feasible, leveraging open-source technologies and low-resource infrastructure for cost efficiency, with a modular web-based design ensuring adaptability and sustainability. It delivers real-time alerts and educational weekly reports, offering a user-friendly solution that promotes network security awareness and practical cybersecurity learning in academic and small organizational settings.

Chapter 4

PROJECT PLAN

This section outlines the overall project plan for WathiqNet: A Web-Based Network Security Monitoring System. It defines the Work Breakdown Structure (WBS), the detailed activity and task list, and presents the corresponding Gantt Chart schedule based on the ProjectLibre plan provided.

4.1 Work Breakdown Structure (WBS)

The Work Breakdown Structure divides the project into five main phases, each containing sub-tasks that represent the logical flow of work:

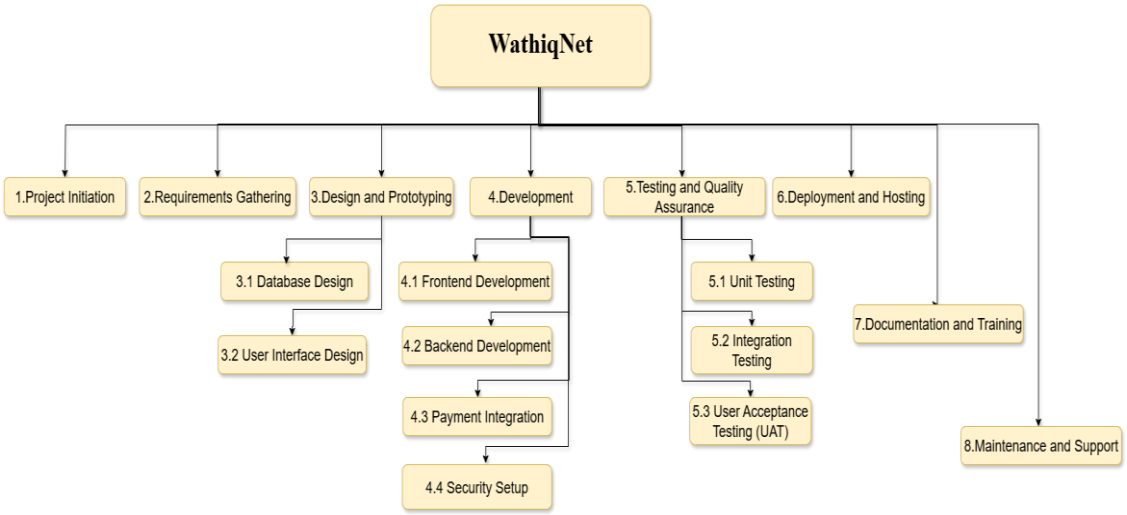


Figure 4.1: Work Breakdown Structure (WBS)

4.2 Activity and Task List

		Name	Duration	Start	Finish
1		1. Project Initiation	2 days?	9/10/25 8:00 AM	9/11/25 5:00 PM
2		2. Requirements Gathering	10 days?	9/12/25 8:00 AM	9/25/25 5:00 PM
3		**3. Design and Prototyping**	13 days?	**9/26/25 8:00 AM**	**10/14/25 5:00 PM**
4		3.1 Database Design	3 days?	9/26/25 8:00 AM	9/30/25 5:00 PM
5		3.2 User Interface Design	10 days?	10/1/25 8:00 AM	10/14/25 5:00 PM
6		**4. Development**	41 days?	**10/15/25 8:00 AM**	**12/10/25 5:00 PM**
7		4.1 Frontend Development	15 days?	10/15/25 8:00 AM	11/4/25 5:00 PM
8		4.2 Backend Development	20 days?	11/5/25 8:00 AM	12/2/25 5:00 PM
9		4.3 Payment Integration	2 days?	12/4/25 8:00 AM	12/5/25 5:00 PM
10		4.4 Security Setup	3 days?	12/6/25 8:00 AM	12/10/25 5:00 PM
11		**5. Testing and Quality Assurance**	6 days?	**12/11/25 8:00 AM**	**12/18/25 5:00 PM**
12		5.1 Unit Testing	2 days?	12/11/25 8:00 AM	12/12/25 5:00 PM
13		5.2 Integration Testing	2 days?	12/13/25 8:00 AM	12/16/25 5:00 PM
14		5.3 User Acceptance Testing (UAT)	2 days?	12/17/25 8:00 AM	12/18/25 5:00 PM
15		6. Deployment and Hosting	10 days?	1/1/26 8:00 AM	1/14/26 5:00 PM
16		7. Documentation and Training	30 days?	1/15/26 8:00 AM	2/25/26 5:00 PM
17		8. Maintenance and Support	10 days?	2/26/26 8:00 AM	3/11/26 5:00 PM

Figure 4.2: Activity and Task List

4.3 Gantt Chart

The Gantt Chart visually represents the project's timeline and dependencies, as prepared in ProjectLibre. Key highlights include:

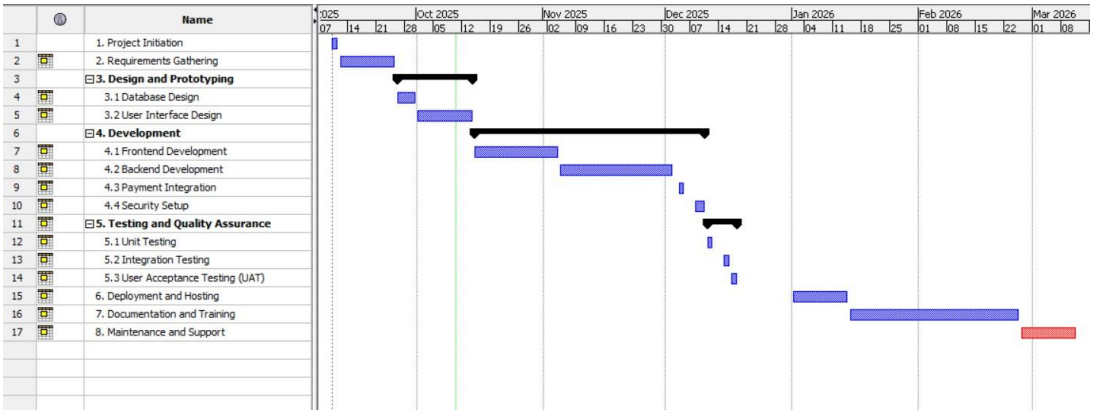


Figure 4.3: Gantt Chart

4.4 Conclusion

The project plan ensures structured and timely execution of WathiqNet. Through clearly defined tasks, dependencies, and milestones, the team can monitor progress and make

adjustments as needed. The combination of the WBS, detailed task list, and Gantt chart offers transparency and accountability, facilitating successful completion of the web-based network security monitoring system within the projected academic timeframe.

Chapter 5

SYSTEM REQUIREMENTS SPECIFICATION (SRS)

This section defines the detailed system requirements for WathiqNet, covering the development methodology, functional and non-functional requirements, hardware and software specifications, and other essential constraints required to ensure successful implementation and operation.

5.1 Development Methodology

WathiqNet adopts the Agile Software Development Methodology, specifically the Iterative–Incremental Model, to ensure flexibility, transparency, and continuous improvement throughout the project lifecycle.

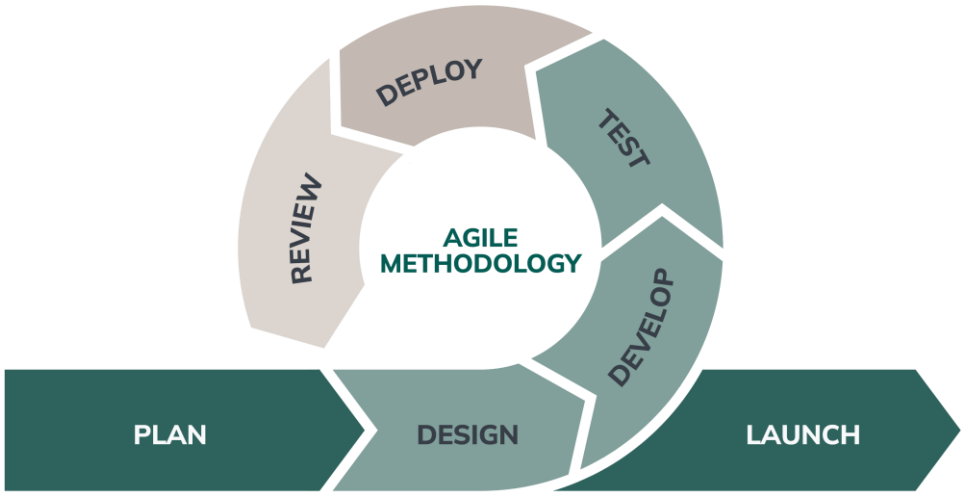


Figure 5.1: Agile Software Development Methodology

5.1.1 Rationale for Choosing Agile

- The system involves continuous refinement of detection rules and dashboard usability, which benefits from Agile's short cycles.

- Stakeholders (team members, supervisor, and potential users) can review prototypes regularly and provide feedback.
- Allows rapid adaptation when functional requirements evolve, especially in network security where rule logic and alerts require frequent adjustments.

5.1.2 Iteration Structure

Each iteration consists of:

1. **Planning:** Identifying functional modules such as log collection, alert engine, reporting, and dashboard.
2. **Design:** Creating mockups, schemas, and workflow diagrams.
3. **Development:** Coding specific features in small increments.
4. **Testing:** Conducting unit tests, integration tests, and usability evaluations after each increment.
5. **Review:** Demonstrations to the supervisor and making improvements before the next cycle.

5.1.3 Benefits for WathiqNet

- Enables early deployment of basic monitoring features.
- Ensures that the dashboard interface evolves based on usability testing.
- Helps fine-tune the rule-based detection mechanism.
- Reduces development risks by breaking down complex logic into smaller tasks.
- Supports collaborative teamwork and parallel development of modules.

5.2 System Requirements Analysis

This section outlines a comprehensive analysis of user needs, system behavior, environmental conditions, and constraints.

5.2.1 Functional Requirements

A. Data Collection

- The system must collect network activity logs from predefined sources such as routers, firewalls, or uploaded CSV/JSON files.
- Logs must include essential fields such as timestamp, source IP, destination IP, protocol, and packet size.
- The system must validate log files to ensure correct format before processing.

B. Detection Engine

- The system must apply rule-based logic to identify:
  - Unusual traffic spikes
  - Repeated failed connection attempts
  - Access to suspicious external IP addresses
  - Unexpected protocol usage
  - Potential scanning or brute-force patterns
- Detection rules must be editable by administrators.
- Each alert must be assigned a severity level: Low, Medium, High, Critical.

C. Alert Management

- Alerts must be logged in the database with timestamp, source, detection cause, and recommendation.
- Users must be able to:
  - View active alerts
  - Filter alerts by severity, time, or type
  - Mark alerts as resolved
  - Add notes to alerts for documentation

D. Dashboard Interface

- The dashboard must display:
  - Total alerts by severity
  - Number of active devices
  - Recent incidents timeline
  - Traffic summaries and charts
- The interface must update in near real-time or after periodic refresh intervals.

E. Reporting Module

- The system must automatically generate weekly PDF reports.
- Reports must include:
  - Summary of detected incidents
  - Severity distribution charts
  - Top sources of suspicious traffic
  - Recommendations for each incident category
- Users must be able to download or email the reports.

F. User & Role Management

- Support at least two roles:
  - Administrator: Manage detection rules, system configuration, users.
  - Viewer: Access dashboard, alerts, and reports only.
- The system must enforce password security policies.

G. Log Storage & Backup

- Logs must be stored in a relational database.
- System must allow exporting logs to CSV or JSON.
- Weekly backups must be automatically created or manually triggered.

5.2.2 Non-Functional Requirements

A. Usability

- The UI must be intuitive and suitable for non-technical users.
- Complex security terms must be accompanied by plain-language explanations.
- The dashboard must use color-coding for severity status.

B. Performance

- The system must process at least 5,000 log entries per minute in small network environments.
- Dashboard pages must load in under 3 seconds under normal conditions.
- Weekly report generation must finish within 10 seconds for a typical dataset.

C. Reliability

- The system must operate continuously without crashing during normal use.
- Database must protect against partial writes.
- Alerts must not be lost during processing.

D. Security

- User passwords must be hashed.
- All data transferred through the web interface must use HTTPS.
- SQL injection, cross-site scripting (XSS), and CSRF protection must be implemented.

E. Maintainability

- Code must be modular and separated into clear components:
  - Data Parser
  - Detection Engine

- Alert Manager
- Report Generator
- Frontend UI

- Documentation must accompany major modules.

F. Portability

- The system must run on Linux, Windows, and macOS servers.
- The application must work with standard web browsers without add-ons.

5.2.3 User Requirements

End-Users Should Be Able To:

- View real-time alerts and network statistics clearly.
- Understand incident descriptions without advanced technical knowledge.
- Download reports easily.
- Browse logs using search and filters.

Administrators Should Be Able To:

- Configure data sources and upload log files.
- Edit detection rules.
- Add or remove users.
- Adjust system thresholds, alerting behavior, and reporting schedule.

5.3 Hardware Requirements Specification

WathiqNet is optimized for lightweight operation; however, minimum specifications ensure stable functionality.

5.3.1 Server Hardware Requirements

Component	Minimum Specification	Recommended Specification
CPU	Dual-Core 2.0 GHz	Quad-Core 3.0 GHz
RAM	4 GB	8–16 GB
Storage	20 GB HDD	50 GB SSD
Network	Standard Ethernet	Gigabit LAN
Power	UPS-backed (optional)	Essential for report systems

Table 5.1: Server Hardware Requirements

5.3.2 Client-Side Hardware Requirements

- Any device (PC/Laptop/Tablet)
- 2 GB RAM minimum
- Display: 1280×720 or higher
- Modern browser compatibility

5.3.3 Additional Hardware Considerations

- Dedicated test router or virtual network environment.
- External storage for backups (optional).

5.4 Software Requirements Specification

5.4.1 Server Software

- Web Framework (choose one):
  - Laravel (PHP)
  - Django (Python)
  - Node.js (Express)
- Database: MySQL / MariaDB
- Web Server: Apache or NGINX

- Operating System: Linux (Ubuntu recommended), Windows Server, or macOS
- Authentication Libraries
- PDF Report Generator (TCPDF, FPDF, or Python ReportLab)

5.4.2 Client Software

- Modern web browser:
  - Chrome
  - Firefox
  - Edge
  - Safari
- PDF viewer for opening weekly reports

5.4.3 Development Tools

- Visual Studio Code or similar editor
- Git version control
- Postman or similar API testing tool
- Virtualization tools (VirtualBox, VMware) for testing logs

5.4.4 Libraries & Dependencies

- Chart.js or ApexCharts for dashboard graphs
- Bootstrap or TailwindCSS for UI layout
- Logging parser modules (Python: pandas; Node.js: csv-parser, etc.)

5.5 Other Requirements Specifications

This section includes additional constraints and operational conditions that influence the system's performance and usage.

5.5.1 Security Requirements

- Password protection and user authentication are mandatory.
- Sensitive data must be encrypted in transit (HTTPS) and protected in the database.
- Only authorized users may modify detection rules or access administrative settings.

5.5.2 Environmental Requirements

- The system must operate within typical indoor server or classroom environments.
- Suitable internet connectivity is required for real-time logging and updates.

5.5.3 Legal and Ethical Requirements

- The system must follow privacy and data protection guidelines by collecting only necessary network information.
- Logs should not store personal content or sensitive user communications.
- Users must be informed about the monitoring scope and purpose.

5.5.4 Operational Requirements

- Weekly reports must be generated automatically without manual intervention.
- The system should provide fallback messages if log sources become unavailable.
- System updates should be possible without deleting existing logs.

5.5.5 Documentation Requirements

- User Guide explaining dashboard usage and alert interpretation.
- Installation Guide for system setup and configuration.
- Developer Documentation for extending rules or modules.

Chapter 6

SYSTEM DESIGN

This section presents the overall system design of WathiqNet, detailing the structure, workflow, and interaction between major system components. It includes the application design and the various unified modeling diagrams that illustrate system behavior and data organization.

6.1 Applications Design

WathiqNet is designed as a web-based application built around three core layers:

1. **Presentation Layer:** Provides the user interface through a responsive web dashboard, enabling users to view alerts, reports, statistics, and device information.
2. **Application/Logic Layer:** Contains the detection engine, alert generation module, report generator, and data processing logic responsible for analyzing logs and triggering events.
3. **Data Layer:** Stores logs, alerts, system configurations, user accounts, and weekly reports in a relational database to ensure persistent and organized data manage- ment.

This layered architecture ensures scalability, modularity, and easy maintenance as the system evolves.

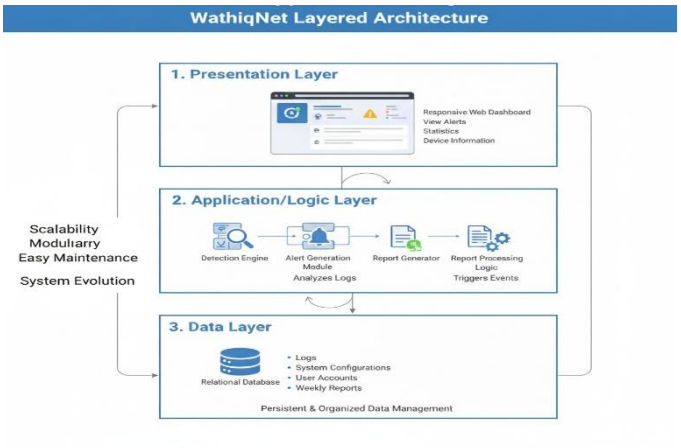


Figure 6.1: Applications Design

6.1.1 Data Flow Diagram (DFD)

DFD Level 0 (Context Diagram)

This diagram shows the WathiqNet System as a single process interacting with two external entities: the User and the External Log Source. It illustrates the system's boundary and its primary data exchanges with the outside world.

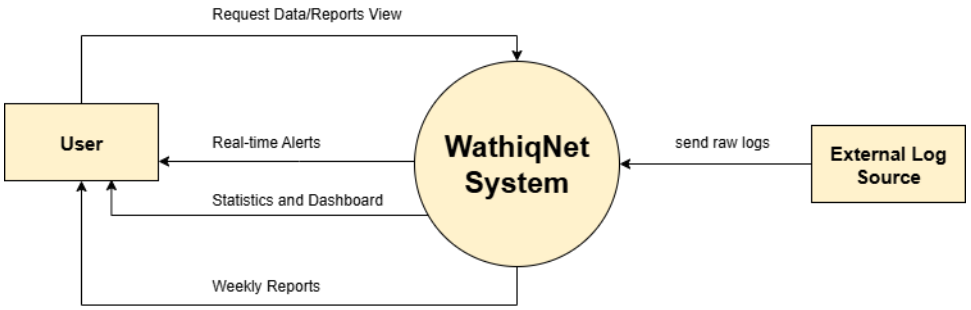


Figure 6.2: DFD Level 0 (Context Diagram)

DFD Level 1

This diagram decomposes the system into several sub-processes, including Collect Logs, Validate & Parse Logs, Apply Detection Rules, Generate Alerts, Generate Reports, and Display Dashboard, showing their interactions with each other, the Database, and the external entities.

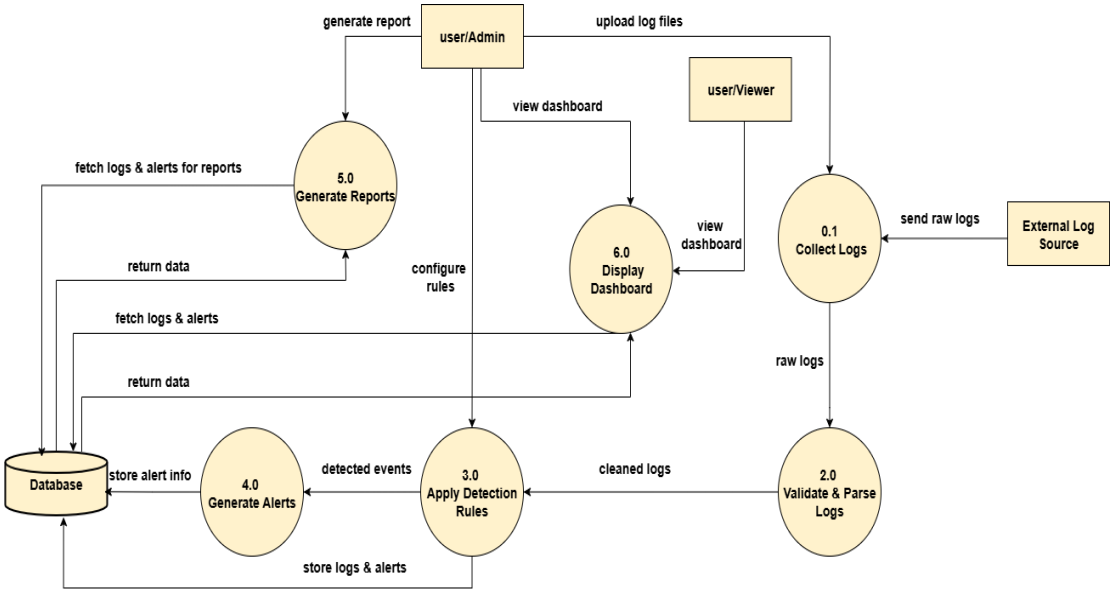


Figure 6.3: DFD Level 1

6.1.2 Class Diagram

The Class Diagram presents the system's main classes—such as User, LogEntry, Alert, Report, and DetectionRule—and their relationships. It defines attributes and methods that form the foundation of WathiqNet's backend architecture.

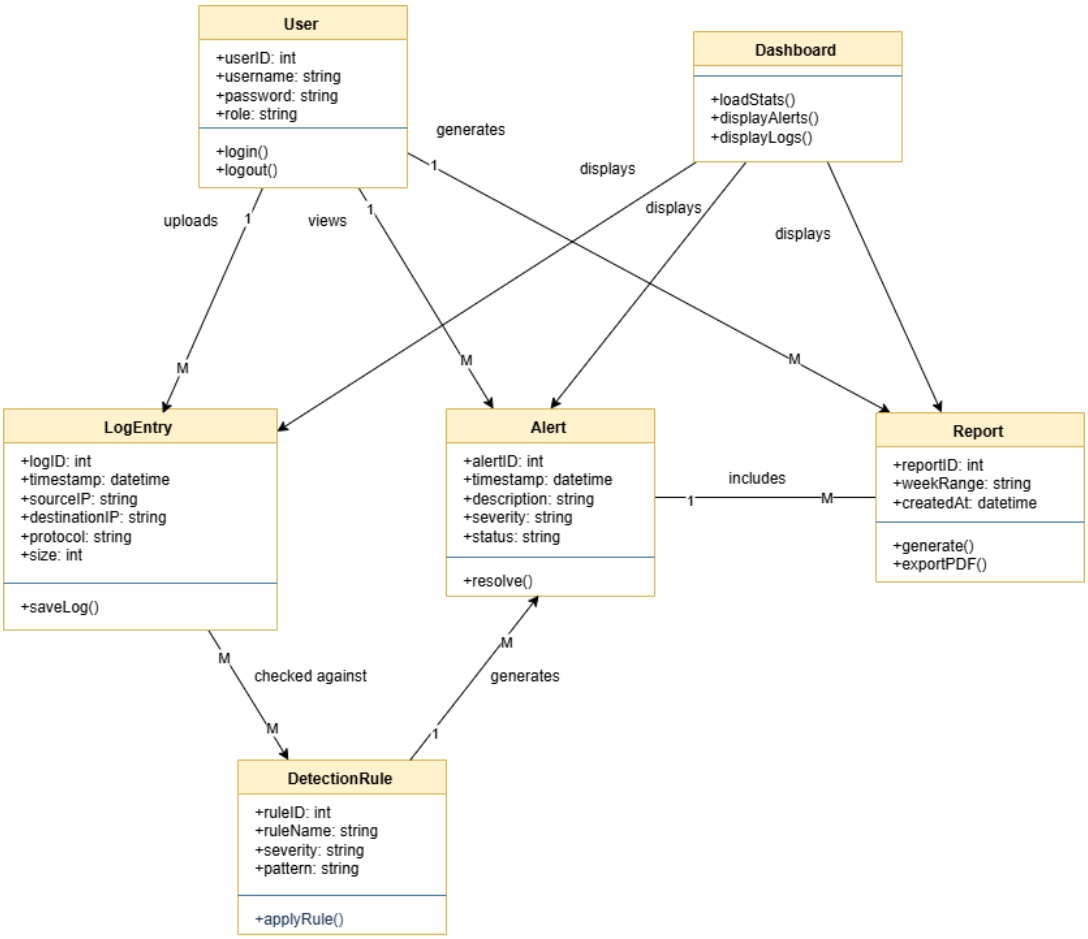


Figure 6.4: Class Diagram

6.1.3 Use Case Diagram

The Use Case Diagram highlights interactions between users (Admin and Viewer) and the system's functions, such as viewing alerts, generating reports, uploading logs, and managing rules. It provides a clear overview of user capabilities within WathiqNet.

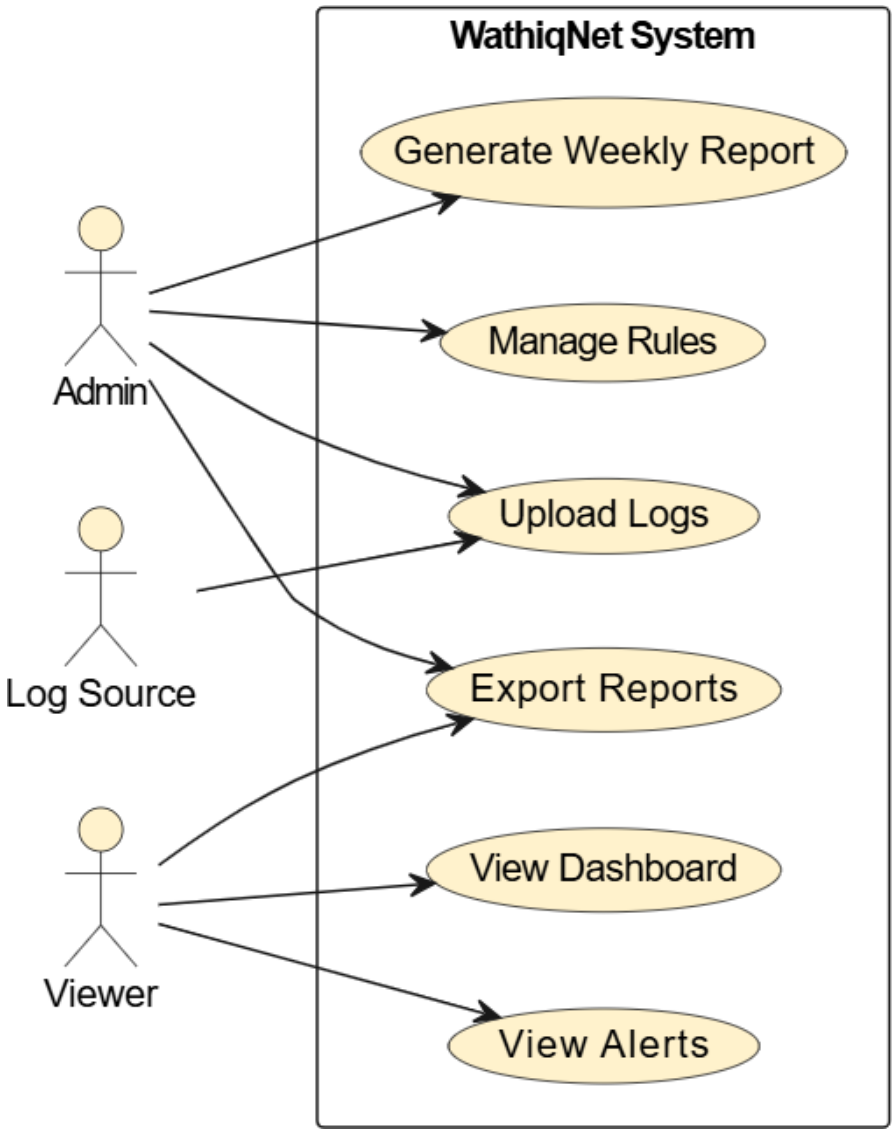


Figure 6.5: Use Case Diagram

6.1.4 Activity Diagram

The Activity Diagram demonstrates the flow of operations for key processes like log analysis and alert generation. It visualizes step-by-step actions from log input to detection, alert creation, and dashboard display.

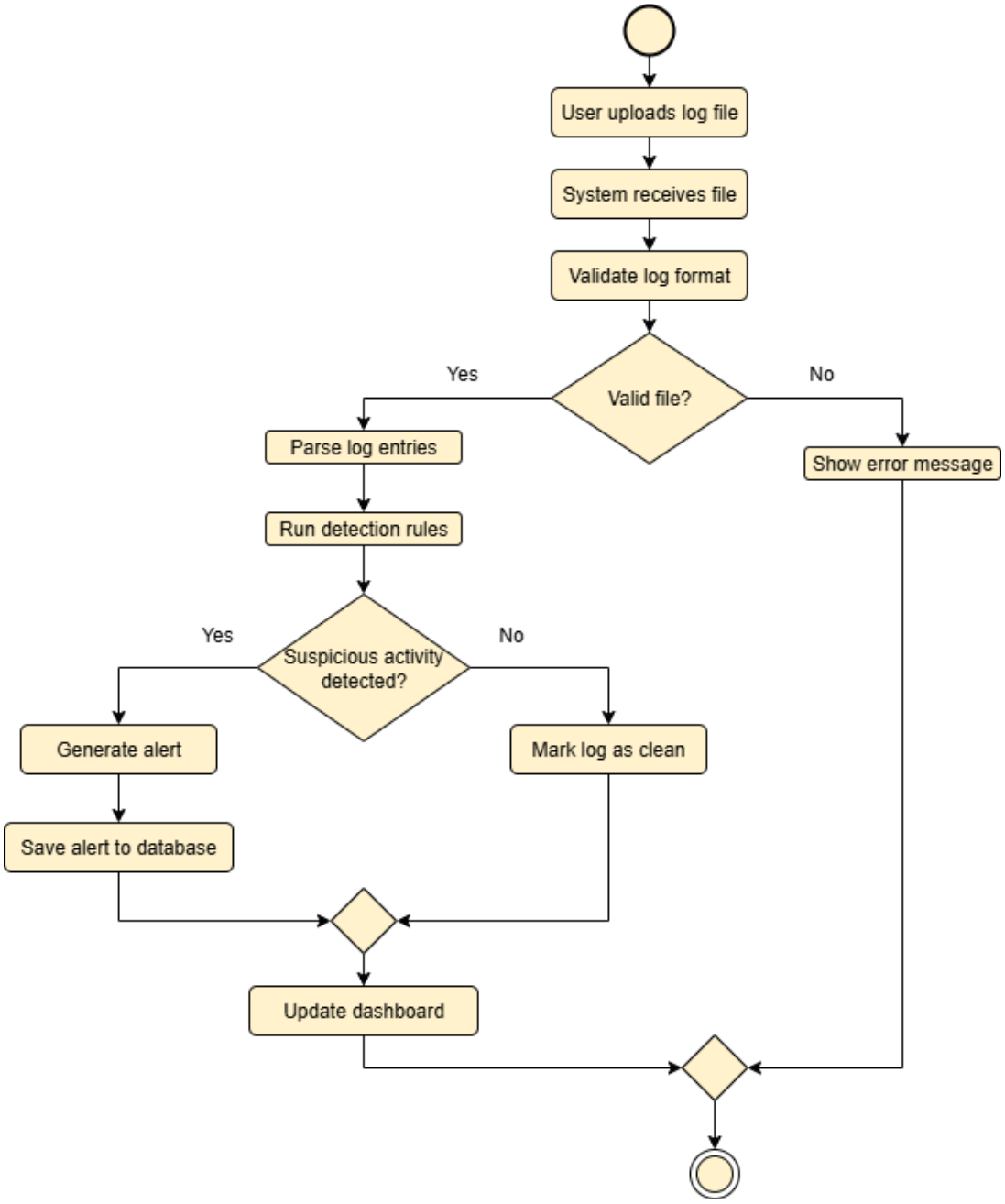


Figure 6.6: Activity Diagram

6.1.5 Sequence Diagrams

The Sequence Diagram shows the chronological communication between system components during log processing, alert creation, and report generation. It explains how messages flow between modules such as the Parser, Detection Engine, Database, and UI.

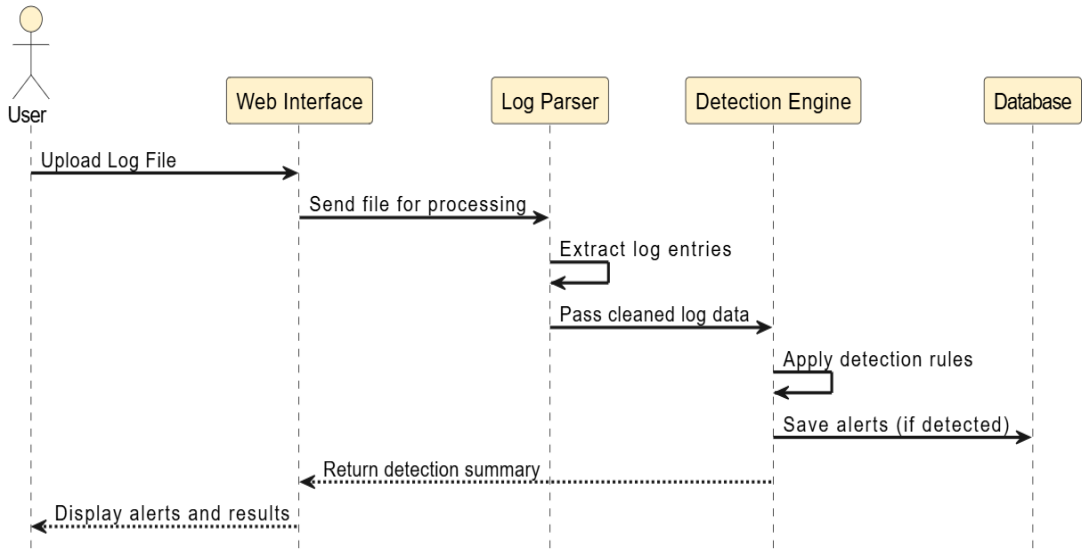


Figure 6.7: Sequence Diagrams

6.1.1 Entity-Relationship Diagram (ERD)

The ERD defines the structure of the database by showing entities like Users, Logs, Alerts, Reports, and Rules, along with their relationships. It ensures that data is logically organized to support efficient retrieval and system performance.

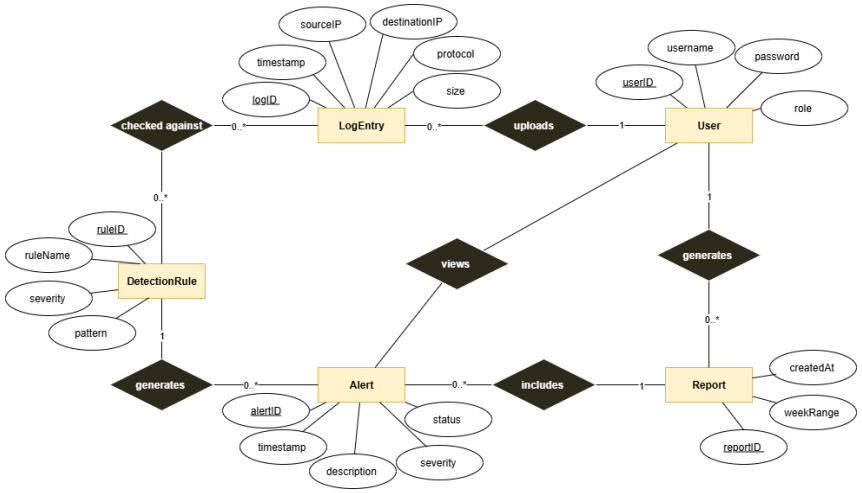


Figure 6.8: Entity-Relationship Diagram (ERD)

Chapter 7

SYSTEM IMPLEMENTATION

This chapter details the practical execution and technical realization of the WathiqNet system, illustrating how conceptual designs and architectural models were translated into a fully functional network security platform. The overarching objective of the implementation phase was to engineer a cohesive, real-time environment capable of seamlessly ingesting network traffic, identifying anomalies through a robust backend detection engine, and presenting actionable intelligence to security personnel. The ensuing sections will outline the systematic construction of the software, beginning with the foundational relational database schema required for secure data management. It then examines the core backend logic, powered by Python and the Flask framework, highlighting the implementation of dynamic threat detection algorithms. Finally, the chapter details the development of the front-end web interfaces, demonstrating how raw network data is visually transformed into an intuitive, role-based interactive dashboard using modern web technologies. Together, these integrated components form a comprehensive and scalable security monitoring solution.

Database Schema Implementation

The WathiqNet application relies on a relational database architecture to securely store user data, network logs, security configurations, and threat alerts. The database is composed of the following primary tables:

1. users Table This table manages the system's authentication and authorization. It stores essential user credentials, including usernames, email addresses, and securely hashed passwords. Crucially, it assigns a role (such as admin or viewer) to enforce role-based access control across the web application. It also tracks metadata like account creation dates and last login timestamps for auditing purposes.

nids | Table name: users | WITHOUT ROWID | STRICT

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	
1	id	INTEGER								NULL
2	username	VARCHAR (80)								NULL
3	email	VARCHAR (120)								NULL
4	password_hash	VARCHAR (256)								NULL
5	role	VARCHAR (20)								NULL
6	created_at	DATETIME								NULL
7	last_login	DATETIME								NULL
8	is_active	BOOLEAN								NULL

2. log_entries Table This is the central repository for all network traffic data ingested into the system. It stores the detailed attributes of every captured packet, including timestamps, source and destination IP addresses, ports, transport protocols (TCP/UDP/ICMP), packet sizes, and transmission flags. This massive dataset acts as the historical record for the *Daily Logs Page* and provides the raw data that the detection engine analyzes for threats.

nids | Table name: log_entries | WITHOUT ROWID | STRICT

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	
1	id	INTEGER								NULL
2	timestamp	DATETIME								NULL
3	source_ip	VARCHAR (45)								NULL
4	destination_ip	VARCHAR (45)								NULL
5	source_port	INTEGER								NULL
6	destination_port	INTEGER								NULL
7	protocol	VARCHAR (20)								NULL
8	packet_size	INTEGER								NULL
9	flags	VARCHAR (50)								NULL
10	action	VARCHAR (20)								NULL
11	device_id	VARCHAR ...								NULL
12	raw_data	TEXT								NULL
13	processed	BOOLEAN								NULL

3. alerts Table This table acts as the system's incident tracking ledger. When the detection engine identifies anomalous behavior based on the network logs, a record is created here. It stores the type of threat (e.g., traffic spike, port scan), its calculated severity (low, medium, high, critical), the IPs involved, and a human-readable description with mitigation recommendations. It also tracks incident management workflows, noting if an alert is resolved and which user resolved it.

nids | Table name: alerts | WITHOUT ROWID | STRICT

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	
1	id	INTEGER								NULL
2	timestamp	DATETIME								NULL
3	alert_type	VARCHAR (50)								NULL
4	severity	VARCHAR (20)								NULL
5	source_ip	VARCHAR (45)								NULL
6	destination_ip	VARCHAR (45)								NULL
7	description	TEXT								NULL
8	recommendation	TEXT								NULL
9	is_resolved	BOOLEAN								NULL
10	resolved_at	DATETIME								NULL
11	resolved_by	INTEGER								NULL
12	notes	TEXT								NULL
13	related_log_ids	TEXT								NULL

4. detection_rules Table This table contains the configurable logic that powers the system's threat detection engine. Instead of hardcoding security rules in the software, they are stored dynamically here. Each row defines a specific rule's parameters, such as packet volume thresholds, evaluation time windows, and assigned severity levels. Administrators can enable, disable, or tweak these rules directly through the web interface, making the system highly adaptable to new threats.

nids | Table name: detection_rules | WITHOUT ROWID | STRICT

	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	
1	id	INTEGER	🔑				🔶			NULL
2	name	VARCHAR (100)					🔶			NULL
3	rule_type	VARCHAR (50)					🔶			NULL
4	description	TEXT								NULL
5	enabled	BOOLEAN								NULL
6	severity	VARCHAR (20)								NULL
7	threshold	INTEGER								NULL
8	time_window	INTEGER								NULL
9	parameters	TEXT								NULL
10	created_at	DATETIME								NULL
11	updated_at	DATETIME								NULL

5. devices Table This table maintains a dynamic inventory of all unique hardware or endpoints detected on the network. The system automatically creates or updates records here whenever a new IP address appears in the log_entries. It tracks when a device was first seen, its most recent activity timestamp (which dictates its online/idle/offline status on the dashboard), and aggregates its total network usage statistics.

nids | Table name: devices | WITHOUT ROWID | STRICT

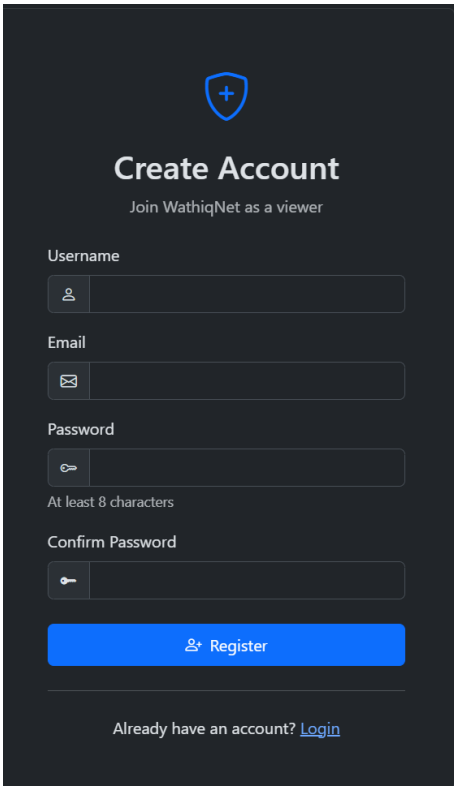
	Name	Data type	Primary Key	Foreign Key	Unique	Check	Not NULL	Collate	Generated	
1	id	INTEGER	🔑				🔶			NULL
2	ip_address	VARCHAR (45)			🧊		🔶			NULL
3	mac_address	VARCHAR (17)								NULL
4	hostname	VARCHAR ...								NULL
5	device_type	VARCHAR (50)								NULL
6	first_seen	DATETIME								NULL
7	last_seen	DATETIME								NULL
8	is_active	BOOLEAN								NULL
9	total_packets	INTEGER								NULL
10	total_bytes	INTEGER								NULL

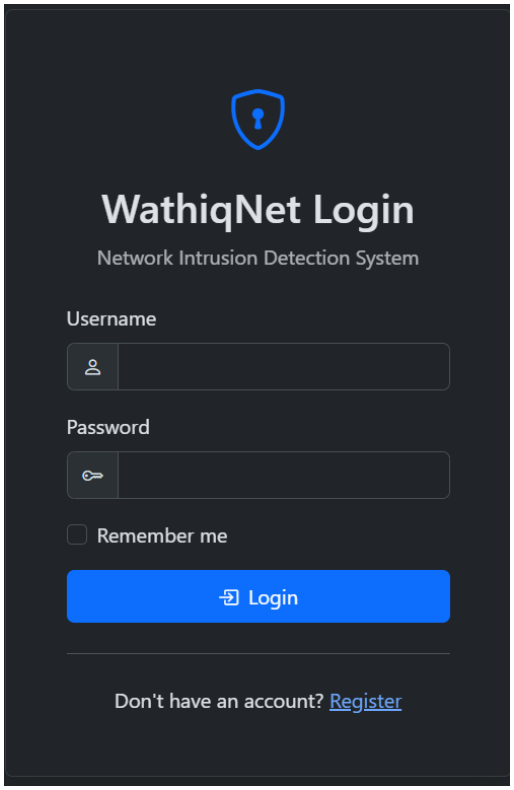
Web Interface Implementation

The front-end web interface of the WathiqNet application serves as the vital bridge between the complex backend threat detection engine and the security analysts operating the system. Designed with usability, clarity, and responsiveness in mind, the web pages are crafted to translate vast amounts of raw, dense network traffic data into intuitive, actionable security insights. Built using a combination of modern web technologies—including HTML5, custom CSS for a modern glass morphism aesthetic, JavaScript for dynamic, real-time charting, and Jinja templating for dynamic server-side rendering—the interface ensures a seamless and engaging user experience across desktop and mobile devices. Furthermore, the entire web application is built on a foundation of strict role-based access control, securely guarding sensitive administrative features while providing administrators and viewers with specialized tools tailored precisely to their operational roles.

User Authentication

- **Login & Registration Pages:** These pages handle user authentication and account creation. They ensure the system is secure by restricting access to authorized personnel only, while supporting different access levels (Administrators with full control, and Viewers for monitoring only).

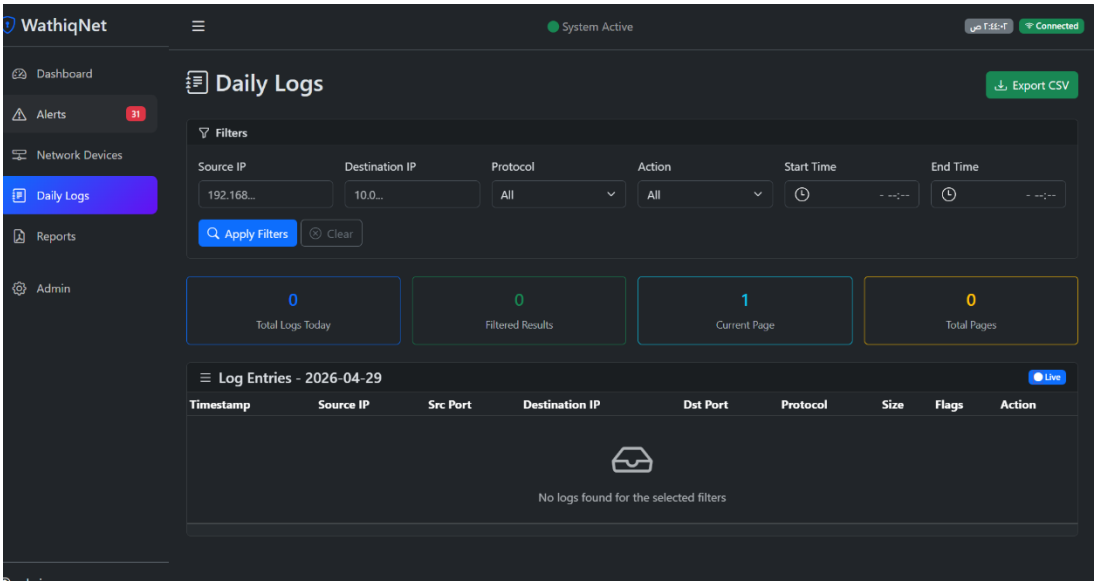




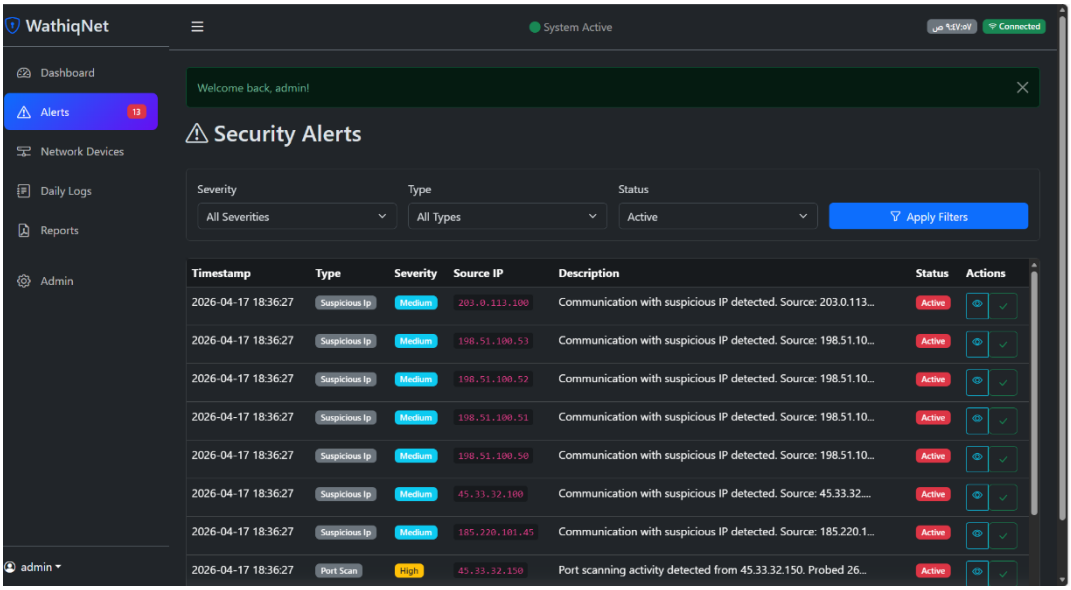
Primary Monitoring Pages

- **Interactive Dashboard (Home):** The central monitoring hub of the application. It provides a high-level, real-time overview of the network's security posture. It features animated statistic cards and dynamic charts (such as alert timelines and severity distributions) to help analysts quickly grasp the current network status.

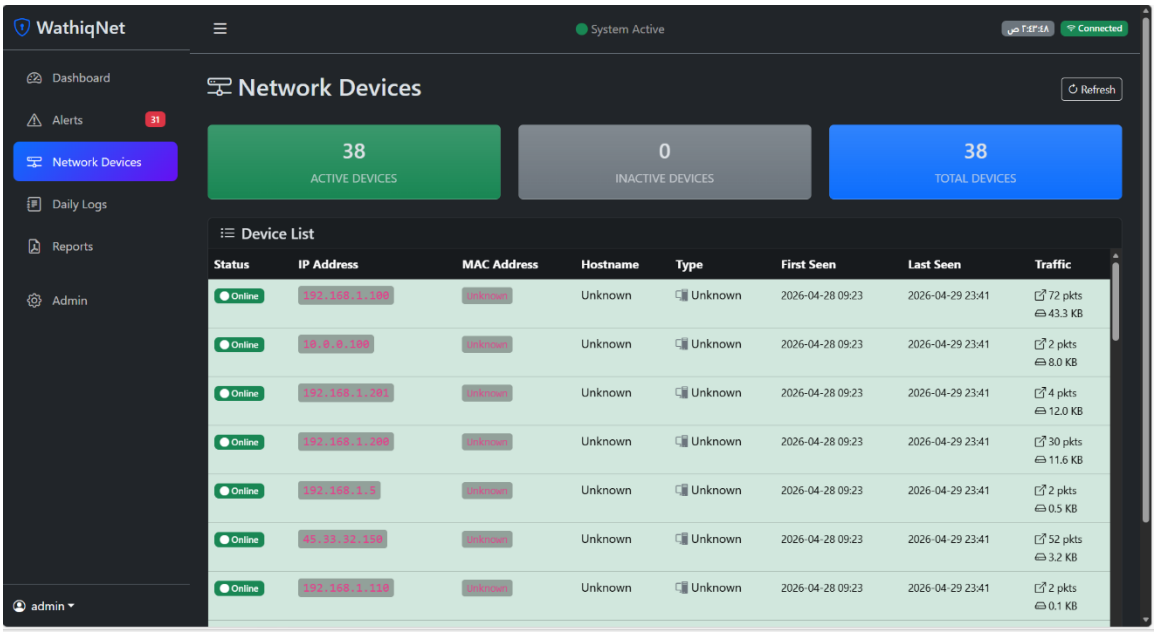
Daily Logs Page: A detailed interface displaying a real-time stream of network traffic records. It equips analysts with advanced data manipulation tools, including filtering (by IP, protocol, port, and time range), sorting, pagination, and the ability to export logs to CSV format for external analysis.



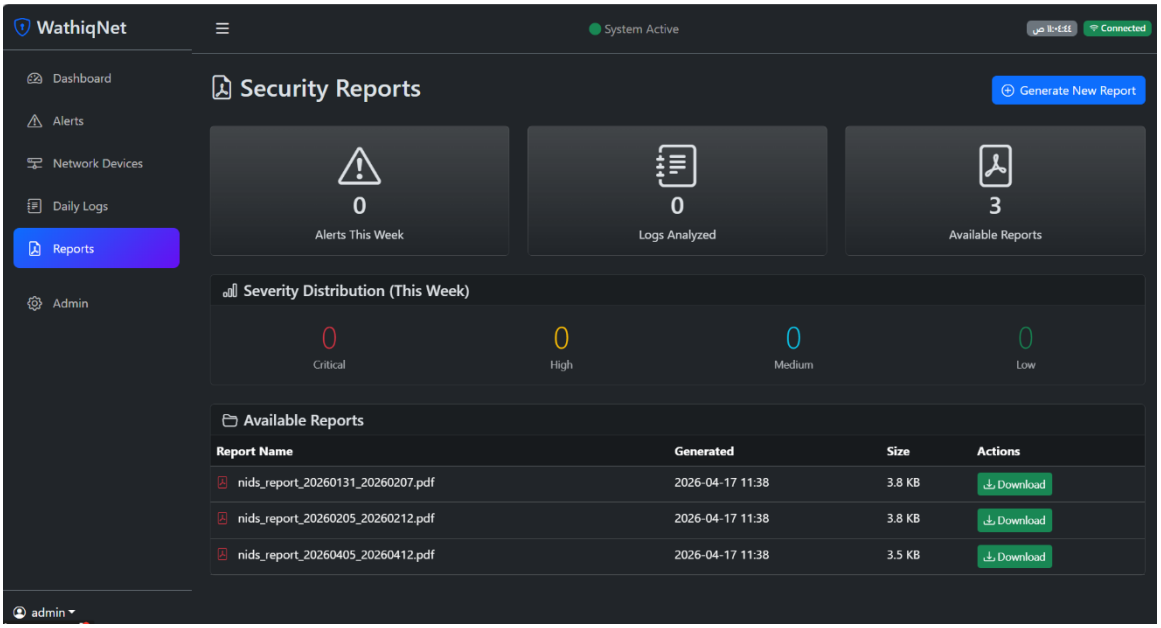
- **Alerts Management Page:** A dedicated workspace for tracking security incidents. It lists all threats flagged by the detection engine, allowing security personnel to filter by severity or status, view specific detection causes and mitigation recommendations, and update the status of alerts (e.g., Acknowledged, Resolved).



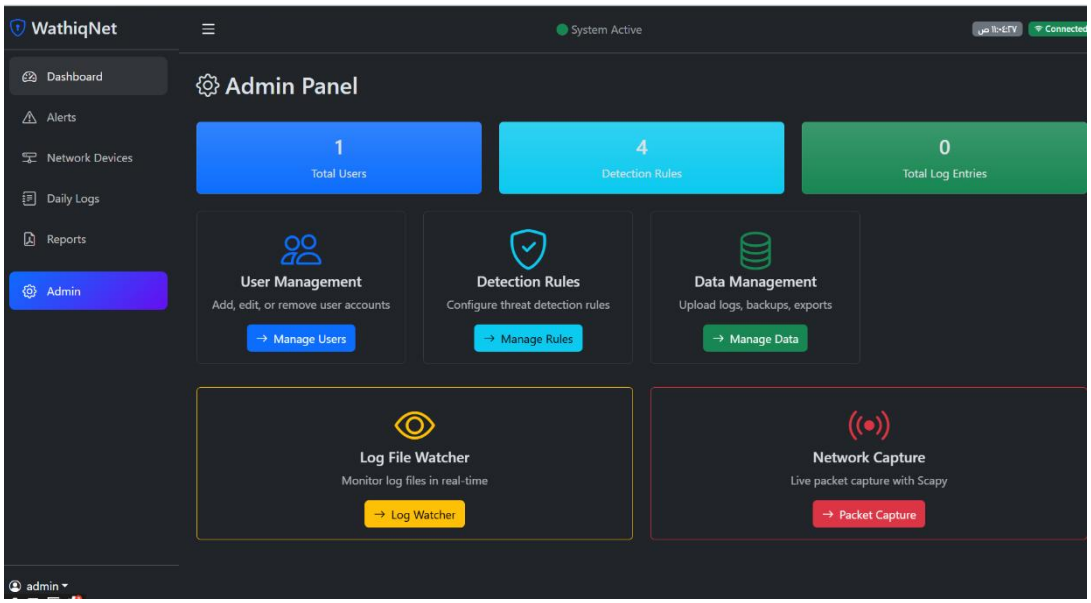
- **Network Devices Page:** An inventory and tracking page that lists all distinct devices detected on the network. It displays their current connection status (Online, Idle, or Offline) and tracks the number of logs and alerts associated with each specific device.



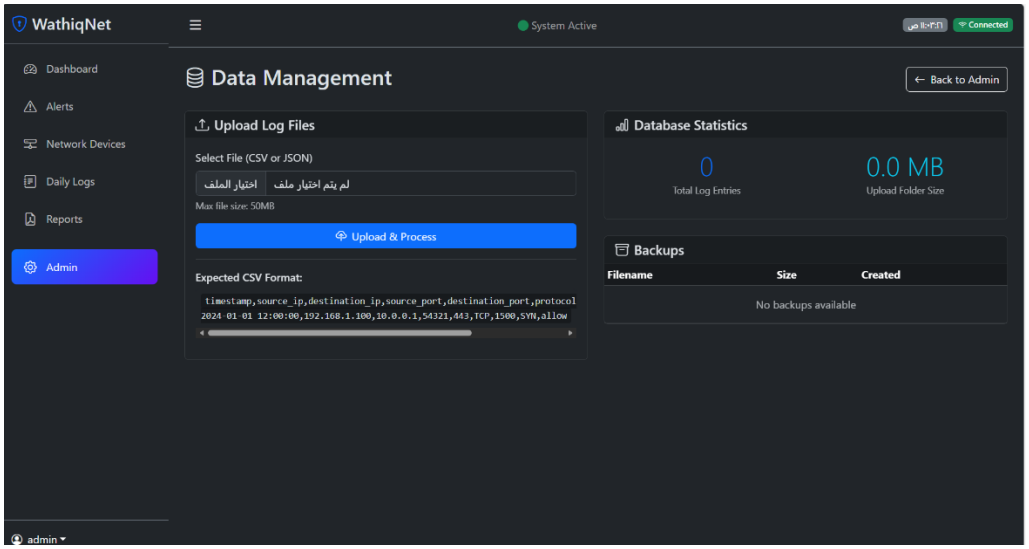
- **Reports Page:** A summary interface where users can view, download, or trigger the generation of comprehensive security reports. These reports summarize network activity and threat trends over specific periods (e.g., weekly reports).



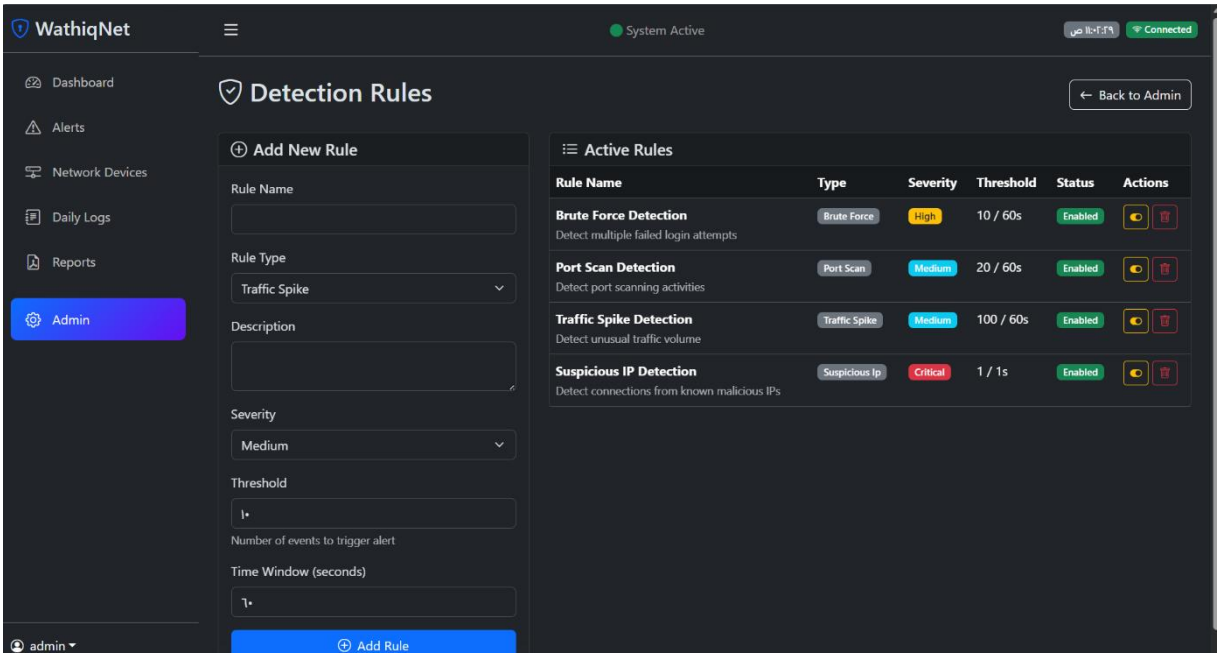
Administrator-Only Pages



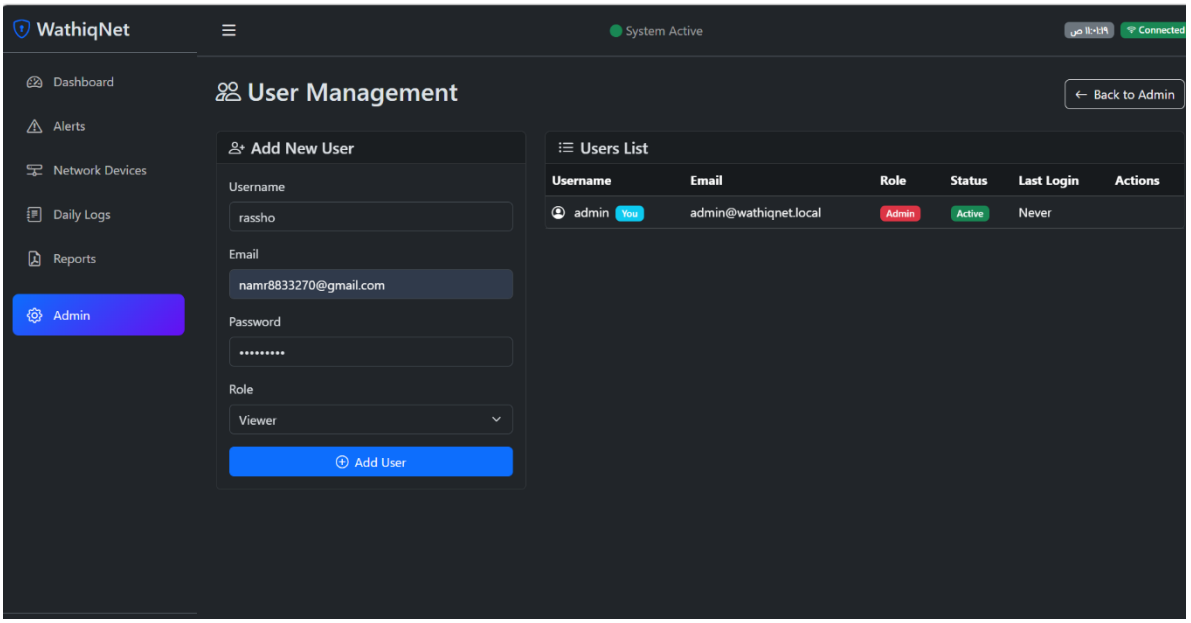
Data Ingestion (Upload Logs): An administrative tool used to manually upload network data files (like CSV logs) into the database. This allows for the historical analysis of data or the testing of the detection engine using sample datasets



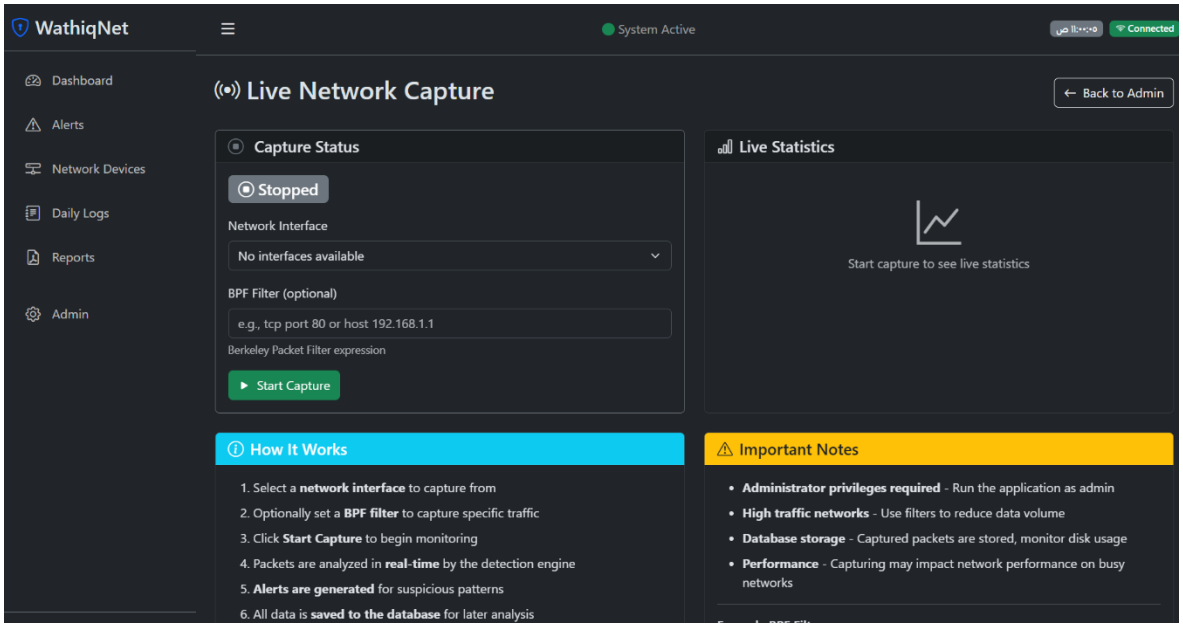
- **Detection Rules Management:** A configuration interface that gives administrators control over the system's brain. From here, admins can activate, deactivate, and adjust the thresholds of various threat detection rules (such as traffic spikes, large packet detection, and suspicious IPs).



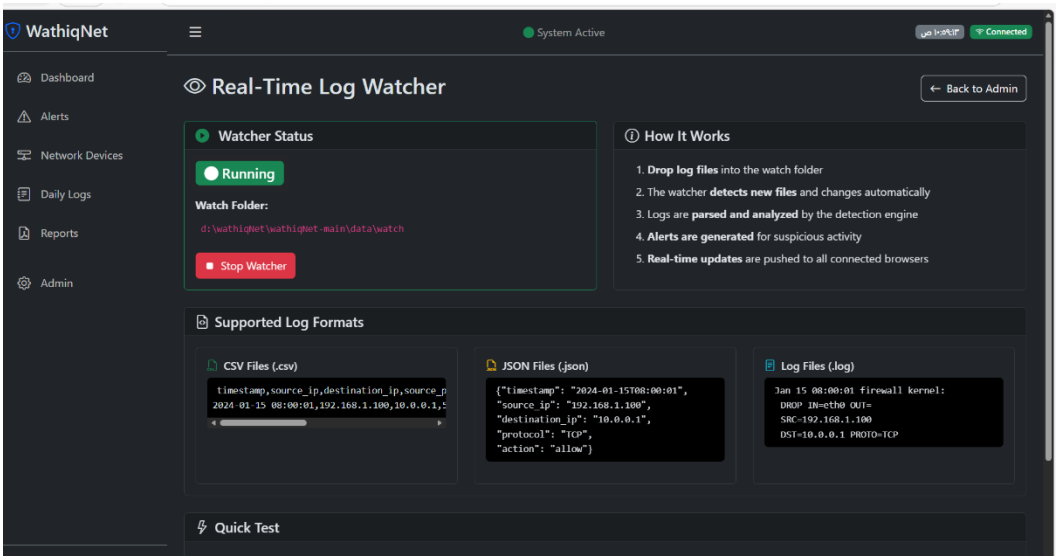
- **User Management:** A control panel for system administrators to oversee the accounts that have access to the platform. Admins can manage user roles and revoke access if necessary.



- **Network Capture Page:** Provides an administrative interface to manage real-time network traffic interception. It allows administrators to start or stop live packet capture on specific network interfaces, feeding live raw data directly into the system's detection engine for immediate threat analysis.



- **Log File Watcher:** An interface configured for automated log file ingestion. Administrators can specify local file paths (such as system, firewall, or router logs) for the application to monitor continuously. As new log entries are written to these external files, the system automatically reads, parses, and processes them in real-time.



Conclusion (Expected Outcomes)

The WathiqNet project is expected to deliver a complete, functional, and user-friendly web-based network security monitoring system designed specifically for small organizations, educational environments, and non-technical users. By integrating log analysis, rule-based threat detection, and automated reporting, the system aims to simplify cybersecurity concepts and make network monitoring accessible to users with limited technical expertise.

From a technical perspective, the system is expected to provide real-time visibility into network activity through a clear and intuitive dashboard that highlights alerts, device behavior, and traffic patterns. The detection engine will identify suspicious events—such as abnormal traffic, unusual connection attempts, and interactions with potentially harmful IP addresses—allowing users to understand risks quickly and respond effectively. Weekly reports will further support users by summarizing incidents, categorizing threats by severity, and offering practical recommendations to improve network hygiene.

Academically, the project demonstrates the feasibility of building a lightweight monitoring platform using open-source tools and web technologies. It highlights how rule-based detection, database design, and structured visualization can form the foundation of a functional security system. The system also serves as an educational model that helps students and beginners understand how monitoring, detection, and reporting work together in cybersecurity.

In practical terms, WathiqNet is expected to enhance security awareness, reduce the likelihood of unnoticed threats, and promote proactive management of small-scale networks. By delivering meaningful alerts and easy-to-understand explanations, the system encourages better decision-making and contributes to safer digital environments. As a result, WathiqNet provides both a technical solution and a learning platform, combining utility, simplicity, and educational value in one integrated system.

References

[1] C. Paulsen and P. Toth, "Small business information security: The fundamentals," National Institute of Standards and Technology, NIST Interagency Report 7621 Rev. 1, November 2016. [Online]. Available: https://doi.org/10.6028/NIST.IR.7621r1

[2] Z. Project. (2025) Quick start guide — book of zeek. [Online]. Available: https://docs.zeek.org/en/master/quickstart.html

[3] Zeek. (2025) Official website. [Online]. Available: https://zeek.org/

[4] Graylog. (2025) Monitoring networks with snort ids/ips. [Online]. Available: https://graylog.org/post/monitoring-networks-with-snort-ids-ips/

[5] Unixmen. (2025) Security onion: Linux distro for ids, nsm & log management. [Online]. Available: https://www.unixmen.com/security-onion-linux-distro-ids-nsm-log-management/